Software Development Tools and Practices PUSL2020

Coursework 2026–2027

Semester: Semester 2

Submission Deadline: TBA

Coursework Type: Group Assignment

Element of Assessment: C1

Module Leader: Ms. Pavithra Subhashini | Mr. Anton Jayakody

E-mail: pavithras@nsbm.ac.lk | anton.j@nsbm.ac.lk

Provisional

1 Coursework

This coursework for PUSL2020 is worth 50% of the module marks. It is made up of two parts (i) a software product and (ii) a report containing your testing strategies and implementation. Students must create their own groups, up to a maximum of six group members. Each group must then decide and agree upon which technology they will use as well as specifying and allocating individual roles, and responsibilities.
Note: Each member should at least Implement one functionality.

2 Project Goal

The goal of this coursework is to design and develop a secure, web-based Project Approval System (PAS) that facilitates the matching of student research projects with academic supervisors. The system will prioritize "Blind Matching" to ensure that project selection is based on technical merit and area alignment, eliminating unconscious bias during the initial selection phase.

3 Project Scenario

In academic institutions, the process of assigning supervisors to final-year projects is often manual and prone to partiality. To ensure a fair and interest-driven allocation, the University requires a system where students can propose innovative ideas and supervisors can select projects that align with their specific expertise.
This coursework involves developing the "Blind-Match" PAS. The platform will allow students to submit project proposals categorized by research areas (e.g., Artificial Intelligence, Web Development, Cybersecurity). Supervisors will browse these proposals without knowing the identity of the students. Once a supervisor confirms interest in a project, the system will "reveal" the identities of both parties, enabling formal collaboration. This system empowers the faculty to maintain high academic standards and ensures students are paired with the most qualified mentors for their specific topics.

4 User Roles and Functional Requirements

4.1 Student (Individual or Group)

Responsibilities: Project ideation, proposal submission, and tracking approval status.
Functional Requirements:

- Secure Login: Individual or Group Lead account.
- Project Submission: Create a new project proposal (Title, Abstract, Technical Stack, and Research Area).
- Manage proposal details (View, Edit, or Withdraw before it is matched).
- Status Tracking:
- View the status of the project (e.g., "Pending," "Under Review," "Matched").
- The Reveal: Once a supervisor confirms the match, the student must be able to see the Supervisor's name and contact details.

Provisional

4.2 Supervisor (Faculty Member)

Responsibilities: Defining expertise, reviewing anonymous proposals, and selecting projects for supervision.
Functional Requirements:

- Secure Login: Faculty account.
- Expertise Management:Select preferred research areas/tags (e.g., "Machine Learning," "Cloud Computing").
- Blind Review Dashboard:Browse a list of available projects filtered by preferred research areas.
- Anonymity Constraint: View project details (Title, Abstract, Tech Stack) *without* seeing the student's name or ID.
- Matching Logic:Express interest in a project.
- Confirm Match: Once confirmed, the system must trigger the "Identity Reveal," showing the student's details to the supervisor and vice versa.

4.3 Module Leader (Coordinator/Admin)

Responsibilities: Oversight of the allocation process, system configuration, and user management.
Functional Requirements:

- Secure Login: Administrative access.
- Research Area Management:Define and manage the list of valid research areas/tags available in the system.
- Allocation Oversight:View a comprehensive dashboard of all matches (Who is paired with whom).
- Manually intervene or reassign projects if necessary.
- User Management:Create and manage accounts for Supervisors and Students.

4.4 System Administrator (Web Master)

Responsibilities: Infrastructure management, database versioning, and security configurations.
Functional Requirements:

- Environment Configuration: Setup of the ASP.NET Core environment and SQL Server connectivity.
- Database Versioning: Implementation of Entity Framework Core Migrations to manage schema changes.
- Security Setup: Implementation of Role-Based Access Control (RBAC) to ensure Students cannot access Supervisor dashboards and vice versa.

5 Deliverables

There are two deliverables, and both should be submitted via DLE.

5.1 Deliverable 1 - Development

- Source Code & Git History: Submission of the complete ASP.NET Core project. Note: The Git commit history must demonstrate professional standards showing a clear progression from CLI initialization to final feature implementation.
- Database: Evidence of EF Core Migrations used to generate the system schema.

Provisional

5.2 Deliverable 2 - Testing and Report

- Comprehensive Testing Suite: Inclusion of Unit tests for matching logic, Integration Tests for database interactions, and documented Functional Tests.
- Technical Report (2,000 words):
  - Explanation of the "Blind-Match" logic and state management.
  - Description of the Version Control strategy used.
  - Critical Analysis of the testing strategy and tools used .
  - Evidence of Professionalism: A log of Git commits and a reflection on solving technical challenges.

5.3 Deliverable 3 – Demonstration Video

- Each group should provide a detailed demonstration of their working product.
- All group members should be visible while the demonstration I been done.
- Each member should introduce themselves and explain their contribution.
- Guidelines for the video:
  - Video Length should be **no longer than 10 minutes**.
  - Video should be uploaded as an unlisted youtube video and the link of the video can be submitted to the DLE.
  - If the unlisted youtube video is flagged as a violation of the “youtube guidelines” students should create a backup link shared through a public onedrive link.
  - Evaluation will be concluded at the 10 minutes mark therefore students should make their submission in the appropriate timeframe.

Provisional

6 Detailed Assessment Criteria

Category and Weighting	Criteria	Marks	LOs
Software Development (50%)	student submissions, supervisor area selection, blind matching logic, and the identity reveal mechanism.	**25**	LO1
	Evidence of professional Git habits, use of Dependency Injection, and "Clean Code" principles.	**15**	LO1, LO4
	Proper use of EF Core Migrations, data validations (Regular Expressions), and secure handling of the "Blind Match" state.	**10**	LO4
Testing and Reporting (50%)	A 2,000-word technical report that is logically structured, uses professional language, and explains the development environment setup.	**15**	LO2
	Successful execution of Unit Tests (logic), Integration Tests (database), and Functional Tests (user journeys). Evidence of Mocking.	**20**	LO3
	A critical analysis of *why* specific testing tools and strategies were chosen. Evaluation of test metrics	**15**	LO2, LO4

Provisional

Marking Rubric:

Part A: Software Development (50 Marks)

Criteria	Fail (<40%)	Pass (40-59%)	Good (60-74%)	Excellent (75%+)
Functional Requirements (25M)	Missing core logic. Blind matching doesn't work. Identity revealed prematurely.	Basic CRUD works. Blind matching is manual or buggy. Many edge cases fail.	All core features work. Blind-to-reveal transition is automated and logical.	Robust, error-free workflow. Exceeds requirements with advanced UI/UX or search filters.
Code Quality & Best Practices (15M)	Single commit history. No Dependency Injection. Spaghetti code.	Inconsistent Git use. Basic DI used. Code is functional but messy. Poor naming.	Clear feature branching. Consistent DI. Clean, modular C# code.	Micro-commits with clear messages. Full separation of concerns. Exemplary use of Middleware.
DB Integration & Data Handling (10M)	Manual DB setup. Hardcoded strings. No Migrations. SQL injection risks.	Basic EF Core use. Inconsistent migrations. Basic validation.	Proper EF Core Migrations history. Robust Regex and Data Annotations. Secure DB context.	Optimized Linq queries. Complex relational mapping. Advanced schema versioning.

Part B: Testing and Reporting (50 Marks)

Criteria	Fail (<40%)	Pass (40-59%)	Good (60-74%)	Excellent (75%+)
Report Clarity & Professionalism (15M)	Unstructured. No screenshots. Generic AI-generated filler text.	Understandable but lacks technical depth. Basic screenshots.	Professional technical report. Logical thread. Specific references to *their* code.	Deep architectural insights. Highly polished. Academic rigor in citations.
Comprehensive Testing (20M)	No tests or tests that don't run.	Basic Unit Tests only. Low coverage (<30%). No Mocking.	Unit + Integration tests. Use of Moq. Coverage >60%.	TDD approach evidenced. High coverage (>80%). Effective Mocking of DB/Services.
Strategy & Analysis (15M)	No justification.	Describes what was tested but not why. Basic analysis.	Critical evaluation of testing tools. Analysis of code coverage results.	Sophisticated analysis. Evaluates trade-offs. Justifies the specific test suite design.

Provisional

7 Academic Offences

See below for excerpt of Section AST10.2. Full details on the academic offences framework and procedures, consult Section AST10 is available here.

Academic offences occur when activity is undertaken which could confer an unfair advantage to any candidate(s) in assessment. The University recognises the following (including any attempt to carry out the actions described) as academic offences, regardless of intent:

a) Copying or paraphrasing of other people's work or ideas into a submitted assessment without full acknowledgement (plagiarism).
b) Unauthorised collaboration of students (or others) in a piece of work (collusion).
c) Making false declarations in an attempt to obtain either modified assessment provisions or special consideration (e.g. of extenuating circumstances).
d) Persuading another member of the University or partner institution (student, staff, or other) to participate in any way in actions which would be in breach of these regulations.
e) Misrepresenting research outcomes and results.
f) Being party to any arrangement which would constitute a breach of these regulations.
g) The inclusion in a piece of assessed work (other than an examination or test) of material which is identical or substantially similar to material which has already been submitted for any other assessment within the University.
h) Any other activity which could confer an unfair advantage to any candidate(s).

8 Submission Type

The report should be a PDF. A zip file containing project work and other assets should be uploaded to the Plymouth Digital Learning Environment (DLE). A youtube unlisted video link should be submitted (please refer to 5.3 section for information).

Provisional